Cybersecurity Internship Assessment Report: HackerSchool CTF Machine

Prepared by: Candidate
Target Organization: TrustNet Secure
Date: October 10, 2026
Submission Channel: career@trustnetsecure.com

1. Executive Summary

This technical assessment report details the execution and complete resolution of the practical challenge machine designated for the TrustNet Secure Cybersecurity Internship selection process. The objective was to perform external/internal reconnaissance, locate security constraints, bypass authentication protocols, and isolate horizontal or vertical elevation vectors within an authorized system framework. Both targeted milestones—uncovering lower-privilege credentials and extracting the administrative root flag—were successfully resolved.

Flag Level	Captured Value	System Location
User Level (goblin)	Authenticated via credentials (ca51)	Local Session Access
Root Level (root)	root-flag{a9dcac68bfa4b4f6319e09fa041e3cb3}	/root/flag.txt

2. Scope and Environment Configuration

- **Target Vulnerable Image:** HackerSchool Virtual Application Appliance (hsbox.qcow2)
- **Active Hypervisor Sandbox:** QEMU/KVM Hardware Emulation Layer Platform
- **Attacking Engine Host:** Kali Linux Operating System (Standard Rolling Kernel Bundle)
- **Network Parameters:** Explicit loopback virtualization framework mapping isolated port traffic rules.

3. Step-by-Step Technical Walkthrough

Phase 1: Storage Layer Analysis & Cryptanalysis

Due to operational constraints within standard user-mode virtual network handshakes causing target interfaces to drop active remote commands (returning a generic *tcpwrapped* or connection reset status), an advanced forensic path was initialized. The raw virtual drive system was safely loaded directly inside the host Kali environment using network block device utilities:

```
sudo modprobe nbd max_part=8
sudo qemu-nbd --connect=/dev/nbd0 hsbox.qcow2
sudo mount /dev/nbd0p1 /mnt
```

Parsing the internal data paths exposed specific clues inside user storage domains, confirming the layout of two active system accounts: **jack** and **goblin**. Documentation recovered from exposed notes highlighted a strict boundary schema applied to the goblin account's access credentials:

- The credential space was limited to exactly 4 structural characters.
- The token composition was constrained entirely to the specific set: **{a, c, 1, 5}**.

By applying permutation evaluation logic, the total possibility matrix was narrowed to only 24 possible configurations (4!). Manual application checks successfully identified the correct matching phrase: **ca51**.

Phase 2: Internal Host Reconnaissance

Utilizing the validated credentials (**goblin:ca51**), an interactive session was authenticated locally on the active shell interface. A review of system notes hidden within workspace directories prompted the evaluation of a historical binary wrapper exploit script named *final.sh*, targeting **CVE-2010-0426** path injection flaws within outdated *sudoedit* binary implementations.

Evaluating account configuration allowances by running the system command tool:

```
sudo -l
```

Exposed an alternative, critical logical flaw built directly inside the active operating parameters configuration file.

Phase 3: Administrative Privilege Escalation

The output returned from the sudo query exposed an unstable access configuration layout rule:

```
User goblin may run the following commands on HackerSchool:
(root) ALL, !/bin/su
```

The system administrators attempted to establish security by applying a blacklist mechanism (using the exclamation mark *!/bin/su* block constraint rule) to prevent a user from executing standard shell transitions. However, because the policy design explicitly awarded global execution authorization parameters across all system paths via the statement **(root) ALL**, the restrictive expression was completely bypassed by calling an alternate execution engine.

Executing the primary system interpreter directly under elevated wrappers immediately broke the structural constraint:

```
sudo /bin/bash
```

This bypassed the blacklist constraint entirely, instantly spawning an absolute administrative session context matching the user string **root@HackerSchool**. Navigating to the directory path unmasked the final objective string:

```
cat /root/flag.txt
root-flag{a9dcac68bfa4b4f6319e09fa041e3cb3}
```

4. Vulnerability Remediation Matrix

Finding 1: Flawed Sudoers Blacklist Configuration

Risk Severity: CRITICAL (CVSS v3.1 Base Score: 9.8)
Impact Analysis: Enables unprivileged accounts assigned generic wildcard access permissions to effortlessly bypass explicit path restrictions, resulting in full control over system environments.
Remediation Action: Remove the global ALL declaration parameters from standard user entities within the configuration file. Transition security architecture models from a weak blacklist architecture to a zero-trust *whitelist matrix* that outlines the exact, isolated binary scripts a worker requires to perform necessary business operations.

Finding 2: Insufficient Password Complexity & Structural Data Disclosure

Risk Severity: HIGH (CVSS v3.1 Base Score: 7.5)
Impact Analysis: Drastically minimizes systemic search spaces for offline brute-forcing loops or credential permutation analysis algorithms.
Remediation Action: Implement enforced password standard parameters via Pluggable Authentication Modules (PAM) to demand longer strings and alphanumeric combinations. Ensure administrative hints or plain text logs regarding configuration parameters are fully scrubbed from public system locations.

5. Operational Disclosures & Strategic Takeaways

• **AI Assistance Statement:** Automated synthesis layouts were utilized to maintain clean professional presentation formats and arrange markdown alignment parameters to comply with tight timeline requirements.
• **Key Technical Takeaway:** Security strategies relying on blacklists are brittle and prone to structural failure. When creating administrative authorization mappings, permissions must always be allocated explicitly via deterministic parameters rather than attempting to filter forbidden actions.